

Availability Refresh Storm Analysis

Real-World Motivation Behind the Distributed Tatkal Booking Engine

Executive Summary

During a real Tatkal booking attempt at approximately 11:00 AM, the booking application repeatedly displayed:

“We are experiencing High Load - Please retry”

instead of returning seat availability information.

The issue occurred while attempting to refresh train availability immediately after the Tatkal booking window opened.

Although the exact internal architecture is unknown, this behavior strongly suggests that one or more backend services responsible for availability queries were unable to process incoming traffic within acceptable response times.

This document analyzes the incident from a backend engineering perspective and explains why Availability Refresh Storms are one of the most challenging scaling problems in reservation systems.

Incident Overview

User Actions

The following sequence occurred during the Tatkal booking window:

1. Opened the application before the booking window.
2. Selected source and destination stations.
3. Searched available trains.
4. Opened a specific train and quota.
5. Repeatedly refreshed seat availability.
6. Attempted to obtain the latest seat count immediately after the booking window opened.

Instead of receiving updated availability information, the system repeatedly returned:

“We are experiencing High Load - Please retry”

Evidence

Screenshot Captured During Incident

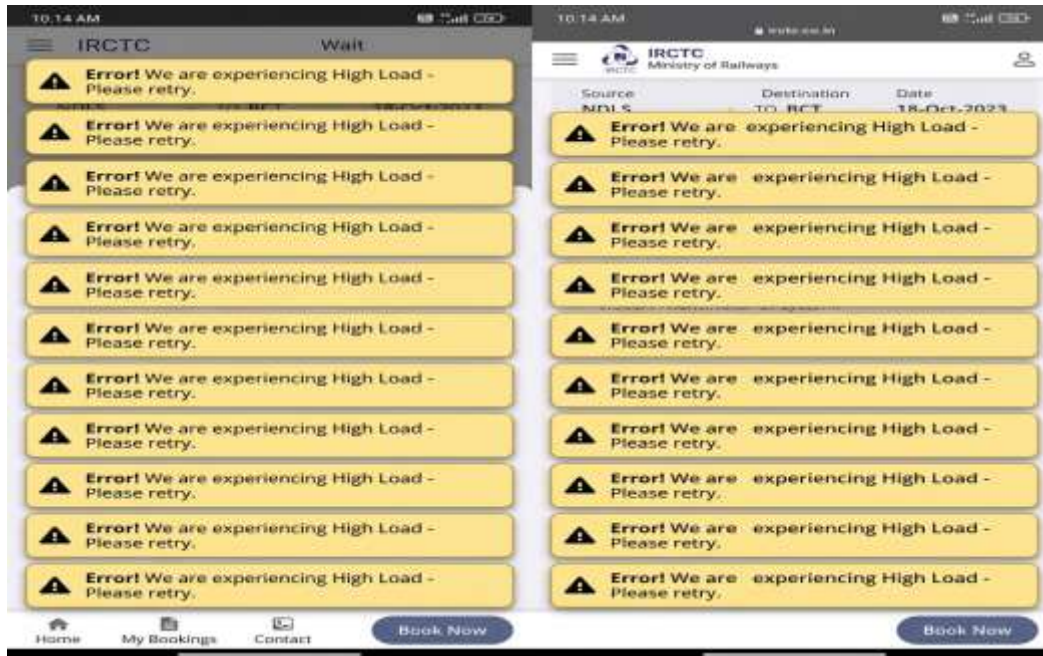


Figure 1: High Load error encountered during Tatkal booking.

Recommended annotations:

- Timestamp
- Train selected
- Quota selected
- Error banner highlighted

Engineering Analysis

What Likely Happened?

At exactly 11:00 AM, thousands of users perform nearly identical actions:

- Search trains
- Open availability screens

- Refresh repeatedly
- Compare seat counts
- Attempt bookings

This creates an extreme burst of read traffic.

The system may normally process:

100–500 requests/sec

During Tatkal opening:

10,000+ requests may arrive within seconds.

This phenomenon is referred to in this project as:

Availability Refresh Storm

Availability Refresh Storm

Definition

An Availability Refresh Storm occurs when a large number of users repeatedly request inventory status information within a very small time window.

Example:

10,000 users × Refresh every 2 seconds

=

5,000 availability requests per second

before booking traffic is even considered.

The availability service becomes the first component under stress.

Why This Problem Is Difficult

Challenge 1 — Sudden Traffic Spike

Normal traffic patterns are predictable.

Tatkal traffic is synchronized.

Instead of traffic gradually increasing, thousands of users arrive simultaneously.

The result:

- CPU spikes
 - Connection pool exhaustion
 - Thread starvation
 - Increased response times
-

Challenge 2 — Database Pressure

If every refresh request hits the primary database:

Availability Service → Database

thousands of times per second,

the database becomes the bottleneck.

Symptoms:

- Slow queries
 - Lock contention
 - Connection exhaustion
 - Timeouts
-

Challenge 3 — Retry Amplification

Users seeing an error rarely wait.

Most users immediately:

- Refresh again
- Retry search
- Reopen availability

This creates additional load.

Example:

10,000 users → error

Each user retries twice

Traffic becomes:

10,000 requests → 30,000 requests

within seconds.

This feedback loop is known as a Retry Storm.

Challenge 4 — Cascading Failures

In a monolithic architecture:

Availability Booking Authentication

often share:

- CPU
- Memory
- Process space

When Availability becomes overloaded:

Booking may also slow down.

Authentication may also slow down.

A failure in one area can impact unrelated functionality.

Questions This Project Attempts To Answer

The Distributed Tatkal Booking Engine was created to investigate the following questions:

1. Can an Availability Service survive a refresh storm?
 2. Can Redis absorb heavy read traffic more effectively than a database?
 3. Can Booking remain operational while Availability is overloaded?
 4. Can Kubernetes automatically scale services under sudden load?
 5. Can service isolation prevent cascading failures?
 6. How much traffic can a local cluster survive before degradation begins?
-

Proposed Experimental Environment

The project simulates:

- One train

- One Tatkal window
- Limited inventory
- Thousands of concurrent users

Technology Stack:

- Node.js
- Express
- MongoDB
- Redis
- Docker
- Kubernetes
- k6 Load Testing

The goal is not to reproduce IRCTC.

The goal is to reproduce and study the architectural challenges exposed by large-scale reservation traffic.

Success Criteria

The simulated architecture will be considered successful if:

- Availability requests remain responsive under load
 - No overselling occurs
 - Booking count never exceeds inventory
 - Auth failures do not affect booking
 - Services scale horizontally using Kubernetes HPA
 - Load tests produce measurable benchmark results
-

Conclusion

The repeated "**High Load - Please retry**" errors observed during a Tatkal booking attempt highlighted the challenge of handling massive bursts of simultaneous availability requests. This real-world incident became a key motivation for building the **Distributed Tatkal Booking Engine** and exploring scalable backend architectures.